

BubbleSort – Klassiker (2019–2025)

Pseudocode (IHK-Stil):

```
Funktion bubbleSort(A: Array<Integer>)  
  n = länge(A)  
  for i = 0 to n-2  
    for j = 0 to n-i-2  
      if A[j] > A[j+1] dann  
        tausche A[j], A[j+1]  
      end if  
    end for  
  end for  
end Funktion
```

Java-Beispiel:

```
public static void bubbleSort(int[] A) {  
  int n = A.length;  
  for (int i = 0; i < n-1; i++) {  
    for (int j = 0; j < n-i-1; j++) {  
      if (A[j] > A[j+1]) {  
        int tmp = A[j];  
        A[j] = A[j+1];  
        A[j+1] = tmp;  
      }  
    }  
  }  
}
```

Erklärung & Beispiel:

Ablauf: Mehrfach benachbarte Paare vergleichen und ggf. tauschen. Schwerster Wert „blubbert“ nach rechts.

Beispiel: $[5,3,1] \rightarrow [3,5,1] \rightarrow [3,1,5] \rightarrow [1,3,5]$.

Komplexität: $O(n^2)$.

Typische Fehler: innere Schleifengrenze falsch; nur einmal tauschen.

Prüfungstipp: Oft als Einstieg gefragt, sehr einfach im Pseudocode.

InsertionSort – Einfügesortieren

Pseudocode (IHK-Stil):

```
Funktion insertionSort(A: Array<Integer>)  
  n = länge(A)  
  for i = 1 to n-1  
    key = A[i]  
    j = i - 1  
    while j >= 0 und A[j] > key  
      A[j+1] = A[j]  
      j = j - 1  
    end while  
    A[j+1] = key  
  end for  
end Funktion
```

Java-Beispiel:

```
public static void insertionSort(int[] A) {  
  int n = A.length;  
  for (int i = 1; i < n; i++) {  
    int key = A[i];  
    int j = i-1;  
    while (j >= 0 && A[j] > key) {  
      A[j+1] = A[j];  
      j--;  
    }  
    A[j+1] = key;  
  }  
}
```

Erklärung & Beispiel:

Ablauf: Elemente werden an der richtigen Stelle im bereits sortierten linken Teil eingefügt.

Beispiel: [5,2,4,1] → nach Einfügen 2 → [2,5,4,1] → usw. → [1,2,4,5].

Komplexität: $O(n^2)$, aber gut für fast sortierte Listen.

Typische Fehler: j nicht richtig dekrementieren; key nicht zurückschreiben.

Prüfungstipp: Beliebt für Prüfungsaufgaben mit kleinem Array.

SelectionSort – Auswahl sortieren

Pseudocode (IHK-Stil):

```
Funktion selectionSort(A: Array<Integer>)  
  n = länge(A)  
  for i = 0 to n-2  
    minIndex = i  
    for j = i+1 to n-1  
      if A[j] < A[minIndex] dann  
        minIndex = j  
      end if  
    end for  
    tausche A[i], A[minIndex]  
  end for  
end Funktion
```

Java-Beispiel:

```
public static void selectionSort(int[] A) {  
  int n = A.length;  
  for (int i = 0; i < n-1; i++) {  
    int minIndex = i;  
    for (int j = i+1; j < n; j++) {  
      if (A[j] < A[minIndex]) minIndex = j;  
    }  
    int tmp = A[i];  
    A[i] = A[minIndex];  
    A[minIndex] = tmp;  
  }  
}
```

Erklärung & Beispiel:

Ablauf: Suche kleinstes Element und tausche es nach vorn.

Beispiel: [64,25,12,22,11] → min=11 → [11,25,12,22,64] → ...

Komplexität: $O(n^2)$.

Typische Fehler: minIndex nicht neu initialisieren; falsche Schleifengrenzen.

Prüfungstipp: Oft als Alternative zu BubbleSort im Vergleich gefragt.

MergeSort – Rekursives Teilen und Mergen

Pseudocode (IHK-Stil):

```
Funktion mergeSort(A: Array<Integer>)  
  if länge(A) ≤ 1 dann return A  
  mitte = länge(A) / 2  
  links = mergeSort(A[0..mitte-1])  
  rechts = mergeSort(A[mitte..ende])  
  return merge(links, rechts)  
end Funktion
```

```
Funktion merge(L: Array<Integer>, R: Array<Integer>) → Array<Integer>  
  i=j=k=0  
  erg = neues Array[Länge(L)+Länge(R)]  
  while i < Länge(L) und j < Länge(R)  
    if L[i] ≤ R[j] dann  
      erg[k]=L[i]; i++  
    else  
      erg[k]=R[j]; j++  
    end if  
    k++  
  end while  
  rest anhängen  
  return erg  
end Funktion
```

Java-Beispiel:

```
public static void mergeSort(int[] A) {  
  if (A.length <= 1) return;  
  int mid = A.length/2;  
  int[] left = Arrays.copyOfRange(A,0,mid);  
  int[] right = Arrays.copyOfRange(A,mid,A.length);  
  mergeSort(left);  
  mergeSort(right);  
  merge(A, left, right);  
}
```

MergeSort – Erklärung & Beispiel

Beispielarray: [38,27,43,3,9,82,10]

1. Teile: [38,27,43] und [3,9,82,10] → weiter teilen

2. Sortiere Teilarrays

3. Führe zusammen mit merge()

Ergebnis: [3,9,10,27,38,43,82]

Komplexität: $O(n \log n)$. Typisch für große Datenmengen.

QuickSort – Rekursiv mit Pivot

```
Funktion quickSort(A: Array<Integer>, low: Integer, high: Integer)
  if low < high dann
    p = partition(A, low, high)
    quickSort(A, low, p-1)
    quickSort(A, p+1, high)
  end if
end Funktion
```

```
Funktion partition(A, low, high) → Integer
  pivot = A[high]
  i = low-1
  for j = low to high-1
    if A[j] ≤ pivot dann
      i++
      tausche A[i], A[j]
    end if
  end for
  tausche A[i+1], A[high]
  return i+1
end Funktion
```

Java-Beispiel:

```
public static void quickSort(int[] A, int low, int high) {
  if (low < high) {
    int p = partition(A, low, high);
    quickSort(A, low, p-1);
    quickSort(A, p+1, high);
  }
}
```

QuickSort – Erklärung & Beispiel

Beispiel: [10,7,8,9,1,5], pivot=5

Partition trennt kleinere/größere Elemente, rekursive Aufrufe sortieren Teilarrays.

Ergebnis: [1,5,7,8,9,10]

Komplexität: $O(n \log n)$ im Schnitt, $O(n^2)$ im Worst-Case.